

EasyFit

Application e-commerce de vêtements de sport

Guillaume BERTIL

Centre Européen de Formation

Titre professionnel Développeur Web et
Web Mobile - Session 2026



Contenu

- Présentation du projet
- Stack technique & architecture
- Maquettes
- Démonstration front-end
- Base de données
- Composants back-end
- Déploiement
- Sécurité
- Difficultés & bilan

Présentation du projet

Contexte

- Projet réalisé dans le cadre de la formation DWWM
- Site e-commerce de vêtements de sport
- Développé de A à Z en autonomie

Objectifs

- Parcourir un catalogue par catégories
- Gérer un panier et passer commande
- Créer un compte et consulter son historique
- Déposer des avis produits
- Interface d'administration back-office

Public cible et limites du projet

Public cible

- Clients: tout utilisateur souhaitant acheter des vêtements de sport en ligne
- Administrateurs: gestionnaires de la boutique via le back-office EasyAdmin

Hors périmètre

- Paiement en ligne (Stripe)
- Gestion de stock liées aux commandes
- Wishlist
- Envoie d'emails transactionnels

Stack technique

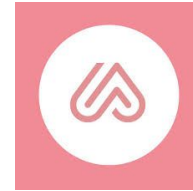
Front-end



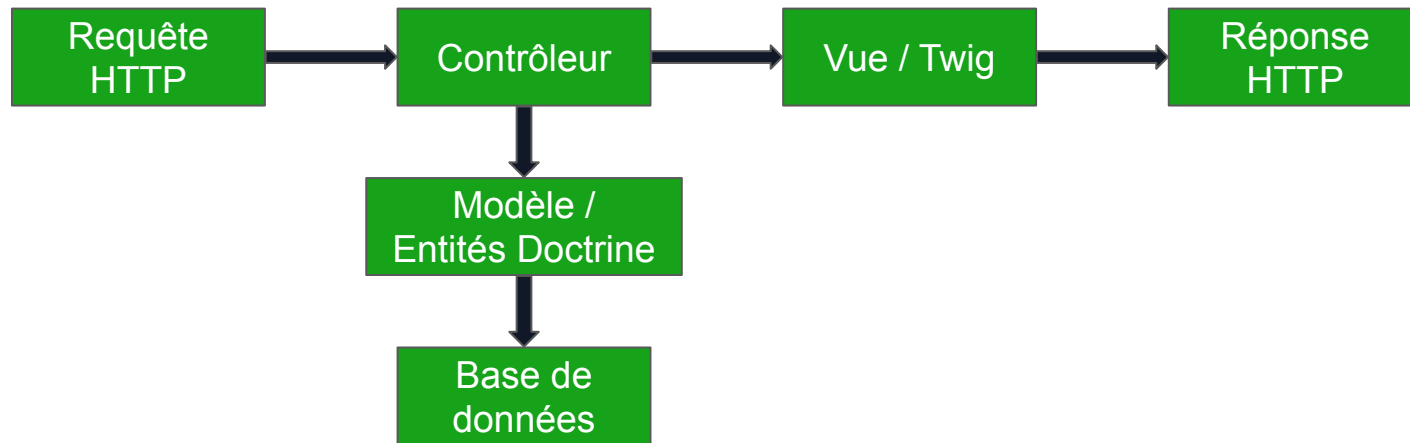
Back-end



Outils



Architecture MVC



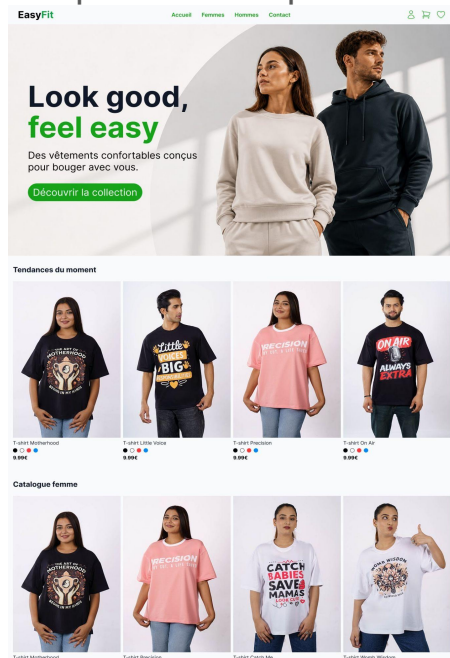
Controller : gère les requêtes HTTP et la logique de routage

Entity / Repository : modèle de données et accès à la base

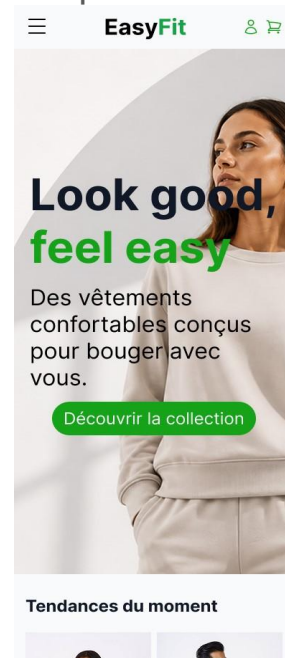
Templates : rendu des vues HTML

Maquettes Figma

Maquette desktop



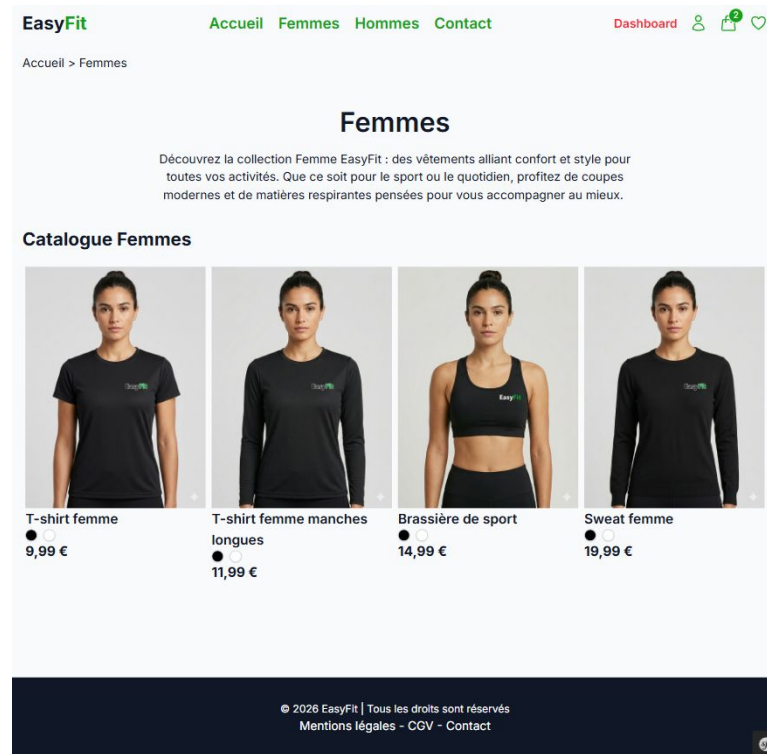
Maquette mobile



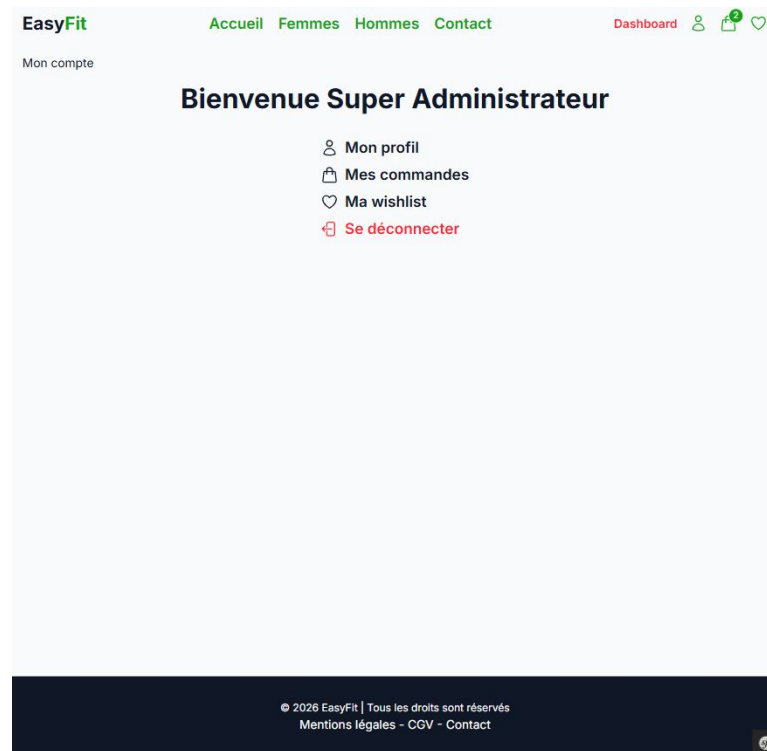
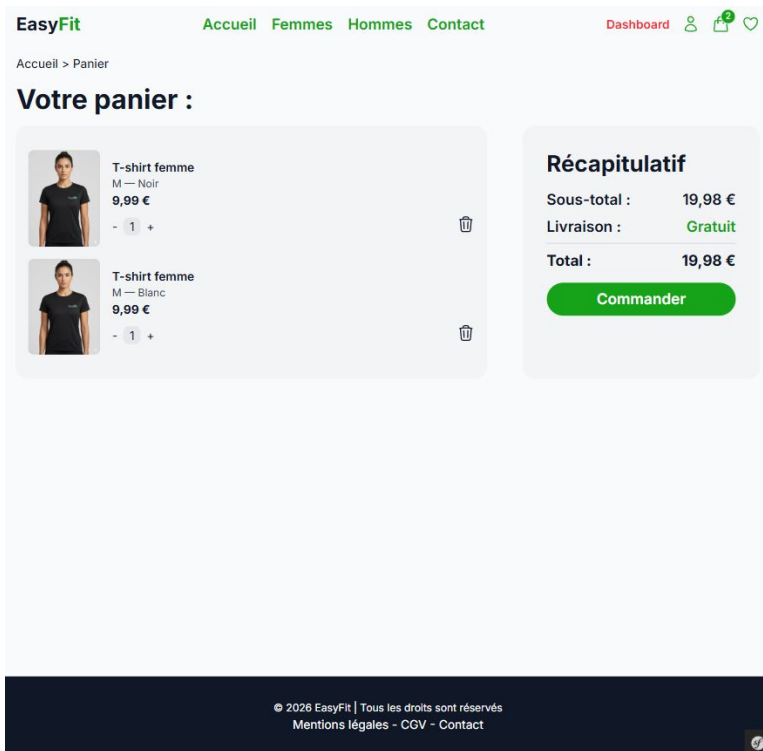
Lien vers la maquette :

<https://www.figma.com/design/za1E5DjYUHPDiS7mL3M4w0/Untitled?node-id=0-1&p=f&t=X3BFS9YUhHcF8mr9-0>

Démonstration front-end

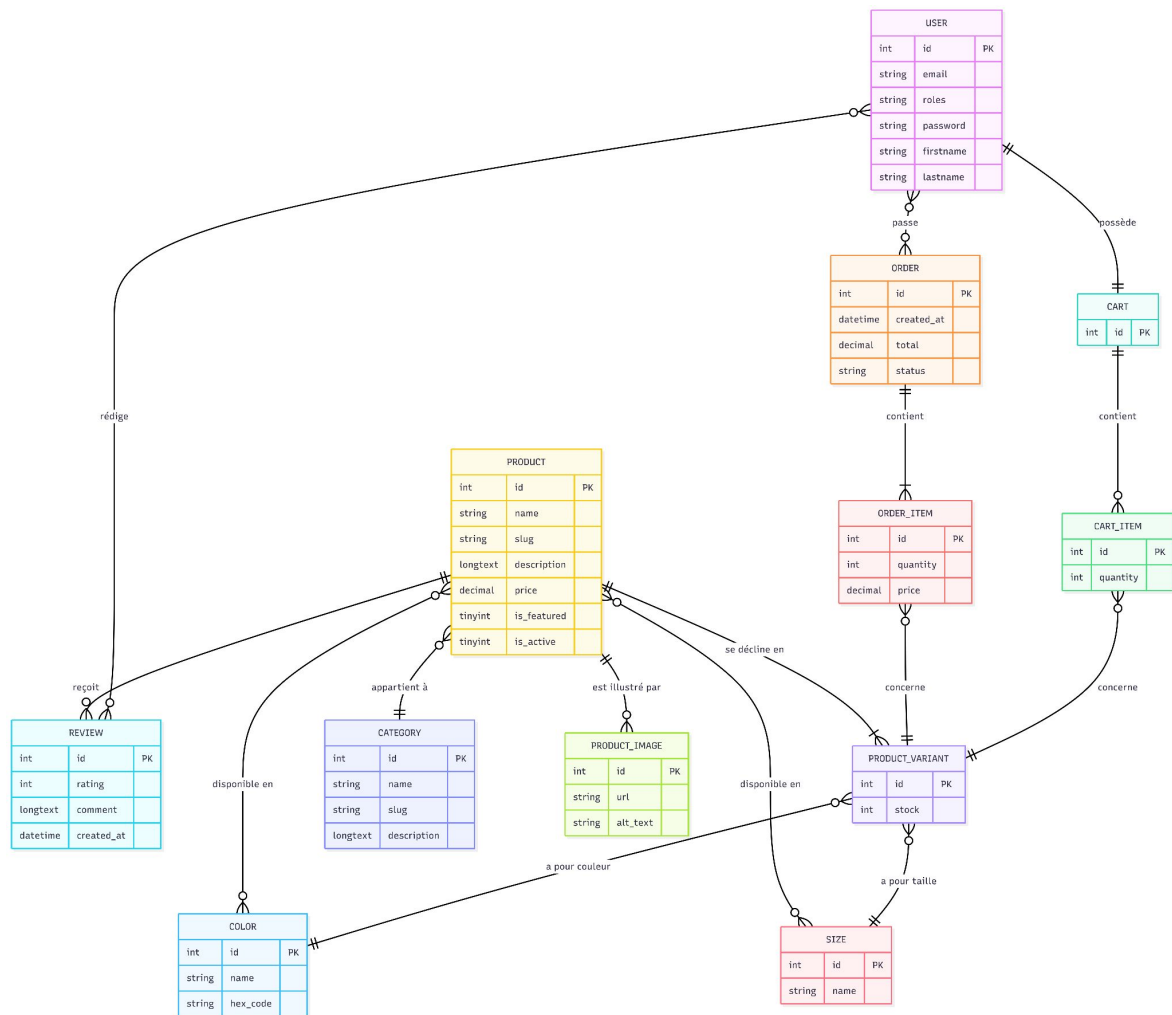


Démonstration front-end



Base de données

- 12 tables métier
- Relations OneToMany, ManyToMany, OneToOne
- Migrations Doctrine pour versionner le schéma
- Clés étrangères et contraintes d'intégrité gérées par Doctrine



Accès aux données - Pattern Repository

- ORM Doctrine - pas de SQL brut
- Pattern Repository pour centraliser les requêtes
- Méthodes natives : findby, findOneBy
- Méthodes personnalisées via QueryBuilder

```
/**
 * Retourne les produits actifs d'une catégorie
 *
 * @return Product[]
 */
public function findActiveByCategory(Category $category): array
{
    return $this->createQueryBuilder('p')
        ->andWhere('p.category = :category')
        ->andWhere('p.isActive = true')
        ->setParameter('category', $category)
        ->getQuery()
        ->getResult();
}

/**
 * Retourne les produits mis en avant
 *
 * @return Product[]
 */
public function findFeatured(): array
{
    return $this->createQueryBuilder('p')
        ->andWhere('p.isFeatured = true')
        ->getQuery()
        ->getResult();
}
```

Composants métier côté serveur

Entités Doctrine

```
/**
 * @var Collection<int, Color>
 */
#[ORM\ManyToMany(targetEntity: Color::class, inversedBy: 'products')]
private Collection $color;

/**
 * @var Collection<int, ProductImage>
 */
#[ORM\OneToMany(targetEntity: ProductImage::class, mappedBy: 'product', cascade: ['remove'])]
private Collection $productImages;
```

Composants métier côté serveur

Formulaires Symfony

```
->add('plainPassword', RepeatedType::class, [
    'type' => PasswordType::class,
    'first_options' => [
        'label' => 'Mot de passe',
        'attr' => [
            'placeholder' => '*****',
        ],
    ],
    'constraints' => [
        new Assert\NotBlank(),
        new Assert\Length([
            'min' => 6,
            'minMessage' => 'Le mot de passe doit contenir au minimum {{ limit }} caractères',
            'max' => 16,
            'maxMessage' => 'Le mot de passe ne doit pas dépasser {{ limit }} caractères'
        ]),
        new Assert\Regex([
            'pattern' => '/[A-Z]/',
            'message' => 'Le mot de passe doit contenir au moins une majuscule'
        ])
    ],
    'hash_property_path' => 'password',
```

Composants métier côté serveur

Enum OrderStatus

```
enum OrderStatus: string
{
    case PENDING = 'pending';
    case PAID = 'paid';
    case SHIPPED = 'shipped';
    case CANCELLED = 'cancelled';

    public function getLabel(): string
    {
        return match ($this) {
            OrderStatus::PENDING => 'En attente de paiement',
            OrderStatus::PAID => 'Payée',
            OrderStatus::SHIPPED => 'Expédiée',
            OrderStatus::CANCELLED => 'Annulée',
        };
    }
}
```

Sécurité

Authentification

- Authentification Symfony Security
- Mot de passe hashé via `hash_property_path`
- Rôles : `ROLE_USER`, `ROLE_ADMIN`
- Routes sécurisées dans `security.yaml`

Bonnes pratiques

- Protection `CSRF` sur tous les formulaires
- Validations des entrées via les contraintes Symfony
- Variables sensibles dans `.env.local` non versionné
- Recommandations `OWASP` et `ANSSI`

Déploiement en production

Hébergement alwaysdata : mutualisé, accès SSH, Apache, MariaDB

Environnement de production : `.env.local`, dépendances `--no-dev`, cache et assets compilés

Base de données créée par migration, données importées depuis le local

Mises à jour par Git : push depuis le local, pull sur le serveur

<https://guillaumb.alwaysdata.net>

Difficultés rencontrées

Tailwind CSS v4 :

configuration déplacée de `tailwind.config.js` vers `@theme` dans le CSS

Badge panier :

extension Twig avec injection de `Security`, le constructeur précédant l'authentification

Mot de passe MySQL en prod :

caractères spéciaux en conflit avec quatre interpréteurs

Migrations non rejouables :

historique désynchronisé, fusionné en une migration unique

Bilan

Bilan

- 8 compétences **DWWM** couvertes
- Application fonctionnelle de bout en bout
- Bonnes pratiques appliquées : Git, PHPDoc, sécurité, accessibilité RGAA
- Projet versionné sur GitHub avec 20+ pull requests
- Application déployée en production sur **alwaysdata**

Évolutions prévues

- Intégration du paiement en ligne (Stripe)
- Gestion du stock liée aux commandes
- Envoi d'emails transactionnels
- Wishlist

Remerciements

Conclusion

EasyFit

Guillaume BERTIL

Lien du dépôt Github : <https://github.com/guillaumbertil/easyfit>

Lien du site déployé : <https://guillaumbert.alwaysdata.net/>